Hybrid Multi-Layer Under-Trained Token Mitigation System (HMUTTMS)

This approach combines detection, cross-validation, and runtime mitigation in a unified architecture that hasn't been implemented in existing research.

Layer 1: Enhanced Detection with Adaptive Lookup

Detection Phase

- Run Magikarp + GlitchMiner in parallel on Model A (target model)
- GlitchMiner provides superior detection (10.7% better than Magikarp alone)
- Aggregate results into a candidate token list with confidence scores

Bloom Filter Pre-screening

- Build a Bloom filter containing all detected under-trained tokens
- Deploy at tokenizer level for instant rejection (sub-microsecond lookup)
- 99%+ of normal tokens pass without further processing

Layer 2: Two-Model Cross-Validation Engine

Validation Architecture

```
text
```

```
Flagged Token → Test in Model A → Test in Model B (superior) → Decision Matrix
```

Decision Matrix with Scoring

- Both fail (A<0.3, B<0.5): CONFIRMED under-trained → Add to permanent blacklist
- A fails, B succeeds (A<0.3, B>0.7): Model-specific weakness → Add to substitution lookup
- Both succeed: False positive → Remove from consideration

Multi-Model Ensemble (Optional Enhancement)

- Use 2-3 diverse superior models (GPT-4, Claude 3.5, Gemma 2)
- Majority voting with weighted confidence
- Expected: 50-70% false positive reduction

Layer 3: Persistent Lookup Table System (Novel Component)

RocksDB-Based Token Registry

text

```
/mitigation_system/
  ├── bloom_filter.bin              # Fast rejection layer
  ├── rocksdb_storage/              # Main persistent storage
  │   ├── blacklist_tokens.db       # Confirmed under-trained
(both models fail)
  │   ├── substitution_map.db       # Model-specific weaknesses
with replacements
  │   ├── metadata.db               # Severity, frequency,
embeddings
  │   └── fallback_rules.db         # Context-aware handling
strategies
```

Why RocksDB:

- 8 microseconds per lookup (126K ops/sec)
- Handles billions of keys
- Persistent storage survives restarts
- Sub-millisecond latency for real-time inference

Lookup Table Structure:

Blacklist Tokens DB

json

```
{
  "token_id": 12345,
  "token_string": "solidgoldmagikarp",
  "status": "BLOCKED",
  "model_a_score": 0.12,
  "model_b_score": 0.38,
  "detection_date": "2025-12-27",
  "action": "reject_and_alert"
}
```

Substitution Map DB (Novel Approach)

```
json
```

```
{
  "token_id": 67890,
  "token_string": "_thethe",
  "status": "SUBSTITUTABLE",
  "model_a_score": 0.25,
  "model_b_score": 0.85,
  "replacement_strategy": {
    "method": "semantic_decomposition",
    "substitute_tokens": [" the", " the"],
    "fallback_embedding": [0.234, -0.456, ...],
  }
}
```

Layer 4: Runtime Mitigation Strategy (Hybrid Novel Approach)

Four-Track Mitigation System:

Track 1: Token Substitution (Novel)

- For tokens that pass Model B but fail Model A
- Replace with semantically equivalent token sequences from substitution map
- Use Model B's successful decomposition as ground truth
- Example: Replace "solidgoldmagikarp" → [" solid", " gold", " magikarp"]

Track 2: Activation-Based Repair (From GlitchProber)

- For tokens in blacklist that must be processed
- Apply GlitchProber's MLP layer rectification
- Adjust intermediate activations to match normal token patterns
- Use cached activation profiles from Model B as repair targets

Track 3: Context-Aware Embedding Injection (Novel)

- For high-severity blacklisted tokens

- Replace token embedding with contextually interpolated embedding
- Use Model B to generate "safe" embedding for that context
- Weighted average of surrounding token embeddings

Track 4: Hard Blocking with Graceful Degradation

- For critical tokens that fail all mitigation attempts
- Block token at tokenizer level before model sees it
- Return informative error with suggested alternatives
- Log incident for continuous improvement

Layer 5: Continuous Learning & Feedback Loop (Novel)

Adaptive System Updates:

```
text

User Input → Bloom Filter → Lookup (RocksDB) → Mitigation →
Monitor Output

↓

Quality Check

↓
                                        ← Feedback Loop ← Update
Strategies
```

Feedback Mechanisms:

- Track mitigation success rates for each token
- If substitution fails repeatedly, upgrade to activation repair
- If activation repair fails, upgrade to hard blocking
- Periodically re-test with newer models as they release

Performance Monitoring:

- Log lookup latency, mitigation overhead
- A/B test different substitution strategies
- Measure model output quality with/without mitigation

Implementation Workflow

Offline (One-Time Setup):

1. Run Magikarp + GlitchMiner detection on Model A
2. Cross-validate with Model B (2-3 superior models)
3. Populate RocksDB with blacklist, substitution maps
4. Build Bloom filter for fast pre-screening

Runtime (Inference Pipeline):

```
text

Input Text → Tokenizer → Bloom Filter Check → RocksDB Lookup →
Mitigation → Model → Output
             ↓            ↓ (0.1µs)          ↓ (8µs)         ↓
(50-200µs)
          Token IDs    99% pass through   If flagged      Apply
strategy
```

Total Overhead: ~10-20 microseconds per token (negligible vs model inference)

Novel Contributions of This Approach

1. First lookup table-based mitigation - No existing work uses persistent storage for token-level intervention
2. Cross-model validation for token classification - Uses superior models to distinguish genuinely broken tokens from model-specific weaknesses
3. Tiered mitigation strategies - Combines substitution, activation repair, embedding injection, and blocking based on severity
4. Real-time inference-compatible - Sub-millisecond lookup enables production deployment
5. Continuous learning architecture - System improves over time through feedback loops

Expected Performance Gains

Based on research analogies:

- Detection accuracy: +25-35% vs Magikarp alone (GlitchMiner + cross-validation)
- False positive reduction: 50-70% fewer incorrect flags

- Mitigation success rate: 60-75% of flagged tokens successfully repaired
- Inference overhead: <1% latency increase
- Scalability: Handles vocabularies of 100K-500K tokens efficiently

This hybrid system addresses the complete pipeline from detection to mitigation while introducing novel components (lookup tables, cross-model validation, tiered strategies) that haven't been explored in existing research on under-trained tokens.

Wardat proposal

BERT model magikarp/glitch miner tool used and collect the bag of undertrained tokens. Then dataset the prompt that pull out the undertrained tokens(training/testing/validation). Then with that fine tune bert model. Then test again. (Fine-tune)